How to evaluate a Langchain RAG system with RAGAs

A guide to evaluating a Langchain RAG system with RAGAs and Weights & Biases.

Brett Young

Created on October 17 | Last edited on October 31

Retrieval-augmented generation systems extend the abilities of large language models by incorporating external data sources, enabling them to generate responses that are more relevant, accurate, and current.

While traditional LLMs rely on the static knowledge embedded during their training, they may struggle with recent events or specialized information. RAG systems address this by dynamically retrieving relevant documents or data from external sources and using that information to augment the model's internal knowledge during generation.

In this tutorial, we will cover how to evaluate a RAG system using a popular framework, called RAGAs.

Jump to the tutorial

Note: You'll sometimes see "RAGAs" with a lowercase "S" or "Ragas" but these are interchangeable. The AS stands for "assessment."

Here's what we'll be covering:

Table of contents

Understanding RAG and evaluations

RAG pipelines are built through a sequence of retrieval, generation, and evaluation steps.

First, documents are embedded into vector representations and stored in a vector database, allowing for efficient similarity-based retrieval. When a user query is submitted, the system searches for the most relevant documents, which are then provided as context to the LLM during response generation. **The final output integrates the retrieved information with the model's internal knowledge**.

Maintaining RAG systems requires continuous updates and monitoring. As external data sources evolve, reindexing ensures that the retrieval component stays aligned with the most current information. Fine-tuning the LLM and adjusting retrieval algorithms may also be needed to maintain or improve precision and relevance.

Regular evaluation helps identify issues such as content drift, irrelevant retrievals, or responses that are inconsistent with available data. RAG systems offer a flexible and effective way to combine internal and external knowledge, but their complexity demands careful design, testing, and ongoing maintenance to ensure consistent performance across a range of applications.

Evaluating these systems has traditionally been challenging due to the need to assess both retrieval and generation components. Retrieval must be judged on how accurately it captures relevant information, while generation is evaluated for relevance, factual correctness, and coherence. Manual evaluation is time-consuming, error-prone, and often requires domain-specific expertise, making it difficult to scale.

This is where **RAGAs** comes in, as it provides an easy-to-use library for both dataset generation, and evaluation. RAGAs simplifies the evaluation process by automating the evaluation using LLM-powered metrics, eliminating the need for human-labeled datasets. It evaluates retrieval quality, measures alignment between responses and retrieved contexts, and identifies potential issues like content drift or hallucinations.

What is RAGAs?

RAGAs is a framework for evaluating Retrieval-Augmented Generation (RAG) pipelines via metric-driven development, ensuring that retrieval and generation components work harmoniously. It simplifies performance assessment and helps generate test datasets with real-world scenario-based queries.

By covering various query types, such as single-hop (simple fact-based) and multi-hop (complex, multi-document) queries, RAGAs thoroughly tests RAG systems under different conditions. It employs a range of metrics to provide insights into both the retrieval and generation processes, assessing how well the system retrieves relevant information, aligns responses with provided data, and addresses user queries meaningfully.

These metrics help identify whether the system avoids introducing errors or relies on irrelevant information during response generation. By measuring aspects of retrieval quality and response accuracy, RAGAs ensures a comprehensive evaluation of the pipeline's performance, highlighting strengths and areas for imp

How RAGAs evaluates RAG pipelines

RAGAs provides a detailed evaluation of RAG pipelines, ensuring both the retrieval and generation stages are performing optimally. What makes RAGAs unique is its use of LLMs to extract, verify, and assess claims, allowing for automated evaluations without needing human-labeled datasets. Below is a breakdown of the key metrics and how they are calculated using LLMs. We will dive into a few of the core metrics that RAGAs supports. Feel free to skip over this section if you are less interested in the mathematical derivations.

Basics of RAGAs metrics

We'll cover some of the main metrics supported by RAGAs. If you aren't a math person, feel free to skip over this section!

Context precision evaluates how well the system ranks relevant information within the top retrieved documents. It ensures that not only relevant documents are retrieved but that the most useful ones appear at the top.

To calculate this, Precision@K is used, which measures the ratio of relevant chunks to the total chunks retrieved at each rank. RAGAs leverages LLMs to identify relevant claims from the retrieved documents by matching them to the user query. For each result, a relevance indicator is assigned—1 if relevant and 0 if not. The overall context precision score is calculated by averaging these weighted precision values across the top K results. This metric ensures that users receive the most useful information in the most accessible way.

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K} (\text{Precision@k} \times v_k)}{\text{Total number of relevant items in the top } K \text{ results}}$$

$$\text{Precision@k} = \frac{\text{true positives@k}}{(\text{true positives@k} + \text{false positives@k})}$$

Where K is the total number of chunks in `retrieved_contexts` and $v_k \in \{0, 1\}$ is the relevance indicator at rank k.

Formula for Context Precision from the RAGAs docs

Context recall focuses on ensuring the completeness of the retrieval process. It measures how well the system retrieves all relevant pieces of information needed to answer the query. Using LLMs, RAGAs breaks down both the retrieved content and

the ground truth (GT) reference into individual claims.

For each GT claim, the system checks whether it can be inferred from the retrieved content. The recall score is then calculated as the proportion of relevant claims found in the retrieved content to the total claims in the GT reference. High context recall is essential in fields like law, healthcare, and research, where missing critical information can lead to serious consequences.

$$\text{context recall} = \frac{|\text{GT claims that can be attributed to context}|}{|\text{Number of claims in GT}|}$$

Formula for Context Recall from the RAGAs docs

Response relevancy measures how closely the generated response aligns with the user's query. This metric ensures that the response is meaningful and directly addresses the user's intent without unnecessary information. RAGAs uses LLMs to generate a set of artificial questions from the system's response.

These questions are reverse-engineered to simulate what the original query might have been. The cosine similarity between these generated questions and the original query is measured to calculate the response relevancy score. A high score means the response effectively addresses the user's needs, making this metric particularly useful in applications like customer service, where concise and relevant answers are crucial.

$$\text{answer relevancy} = \frac{1}{N}\sum_{i=1}^{N} cos(E_{g_i}, E_o)$$

$$\text{answer relevancy} = \frac{1}{N}\sum_{i=1}^{N} \frac{E_{g_i} \cdot E_o}{\|E_{g_i}\|\|E_o\|}$$

Formula for Response Relevancy from the RAGAs docs

Faithfulness ensures that the generated response remains consistent with the retrieved information and avoids hallucinations or contradictions. RAGAs breaks down the generated response into individual claims using LLMs. Each claim is cross-checked against the retrieved context to determine if it can be inferred from the data. For each claim, the system provides a verdict on whether it is supported by the retrieved content. The faithfulness score is calculated as the proportion of verified

claims to the total number of claims in the response. This metric is critical in fields like journalism, finance, and science, where factual consistency is essential.

$$\text{Faithfulness score} = \frac{|\text{Number of claims in the generated answer that can be inferred fror}}{|\text{Total number of claims in the generated answer}|}$$

Formula for Faithfulness Score from the RAGAs docs

Noise sensitivity measures how often a system makes errors by generating incorrect responses, whether based on relevant or irrelevant retrieved documents. A lower noise sensitivity score indicates better performance, with values ranging between 0 and 1. This metric is crucial to ensure the reliability of the RAG system when exposed to potentially misleading information.

To calculate this score, the system checks every claim in the generated response against the ground truth to determine whether it is accurate and whether it aligns with the relevant retrieved contexts. An ideal system would have all claims supported by relevant contexts, minimizing incorrect claims. The equation for noise sensitivity is:

$$\text{noise sensitivity (relevant)} = \frac{|\text{Total number of incorrect claims in response}|}{|\text{Total number of claims in the response}|}$$

Formula for Noise Sensitivity from the RAGAs docs

Factual correctness is a metric used to compare and evaluate the factual alignment between a generated response and a reference answer. This metric helps determine how accurately the generated output reflects the intended information. In RAGAs, factual correctness is computed by first breaking down both the response and the reference into individual claims using LLMs. Each claim is then compared to assess how well the generated content matches the reference.

Factual correctness relies on calculating true positives (TP), false positives (FP), and false negatives (FN). Precision, recall, and F1 scores are derived from these values to quantify the model's factual overlap with the reference data. Higher scores indicate better alignment, reducing the risk of incorrect or misleading outputs. The default mode for this metric is the F1 score, but precision and recall can also be used for this metric.

$$\text{True Positive (TP)} = \text{Number of claims in response that are present in reference}$$

True Positive (TP) = Number of claims in response that are present in reference

$$\text{False Positive (FP)} = \text{Number of claims in response that are not present in reference}$$

$$\text{False Negative (FN)} = \text{Number of claims in reference that are not present in response}$$

The formula for calculating precision, recall, and F1 score is as follows:

$$\text{Precision} = \frac{TP}{(TP + FP)}$$

$$\text{Recall} = \frac{TP}{(TP + FN)}$$

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{(\text{Precision} + \text{Recall})}$$

Formula for Factual Correctness from the RAGAs docs

The tutorial: Evaluating LangChain RAG Systems with RAGAs

Now, we will dive into the tutorial section, where we will build a RAG pipeline using a few different models, and use RAGAs to evaluate the performance of the two systems!

Step one: Setting up our API Keys with a .env File

Before getting into the code, you need to set up your API keys to ensure that the required services are accessible. These keys provide access to OpenAI's GPT models, Anthropic models, and Cohere embeddings. To manage these keys securely, a `.env` file is used, which keeps sensitive information separate from the code.

To begin, create a new file in your project directory and simply name it `.env`. Open this file with a text editor and add your API keys following the correct format. For example, you might have entries like `OPENAI_API_KEY=your-openai-api-key-here` or `COHERE_API_KEY=your-cohere-api-key-here`. These are placeholders, so you will need to replace them with your actual API keys. Once the keys are added, save the file and ensure it is stored in the same directory where your script or project is located.

The following scripts will read the `.env` file using the `python-dotenv` package, which loads the keys into the environment. When the code executes, it retrieves the keys through calls like `os.getenv("OPENAI_API_KEY")` to ensure the necessary services are accessible.

Here's a sample of what the `.env` file will look like:

```
OPENAI_API_KEY=your-openai-api-key-here
ANTHROPIC_API_KEY=your-anthropic-api-key-here
COHERE_API_KEY=your-cohere-api-key-here
```

Step two: Generating test data for evaluation

Before evaluating a RAG pipeline, we need to prepare a test dataset that simulates real-world queries and interactions with external documents. This step involves loading relevant documents, splitting them into manageable chunks, and generating a diverse range of queries. Given these queries, we can test how well the RAG system can retrieve relevant content and produce responses that align with the ground truth answers.

I chose to use a section of documentation within the W&B Weave GitHub repository as documents for the system, to simulate a scenario where a developer asks questions about the library, and the RAG system is tasked with answering the query. When creating test data, the goal is to create a set of realistic queries, including both **"specific"** and **"comparative abstract queries"**, which will allow us to assess the system across multiple dimensions. With this setup, we can comprehensively evaluate how well the pipeline handles different query types, including straightforward fact-based questions and more abstract comparisons that require reasoning over multiple sources.

We'll use LangChain to load and split the documents, and we'll wrap an OpenAI GPT-4 model with the `LangchainLLMWrapper` to generate test queries. The test set will be generated with RAGAs' `TestsetGenerator`, allowing for automated query creation without requiring manual input. This approach not only saves time but also ensures that the dataset covers a wide range of query patterns—key for thorough pipeline evaluation.

By the end of this process, we'll have a 30-query test dataset, saved as a CSV file, which can be used to measure the performance of the RAG system in later stages. This includes testing both the system's ability to retrieve relevant documents and

how accurately it integrates retrieved content into its responses.

Here's the code:

```
import os
import nest_asyncio
import pandas as pd
from dotenv import load_dotenv
from langchain_community.document_loaders import DirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from ragas.llms import LangchainLLMWrapper
from langchain_openai import ChatOpenAI
from ragas.testset import TestsetGenerator
from ragas.dataset_schema import EvaluationDataset
from ragas.testset.synthesizers import SpecificQuerySynthesizer, ComparativeAbstrac
# Apply nest_asyncio to avoid event loop issues
nest_asyncio.apply()

# Load OpenAI API key from environment variables or .env file
load_dotenv()  # Ensure you have a .env file with OPENAI_API_KEY
openai_api_key = os.getenv("OPENAI_API_KEY")

# Verify if the key was loaded correctly
if openai_api_key is None:
    raise ValueError("OpenAI API Key not found. Please ensure you have a .env file

# Check if the Weave repository already exists; if not, download it using sparse ch
repo_dir = "weave_docs"
if not os.path.exists(repo_dir):
    os.system(f"git init {repo_dir}")
    os.chdir(repo_dir)
    os.system("git remote add origin https://github.com/wandb/weave.git")
    os.system("git sparse-checkout init --cone")
    os.system("git sparse-checkout set docs/docs/guides/tracking")
    os.system("git pull origin master")
    os.chdir("..")
else:
    print(f"{repo_dir} already exists, skipping download.")

path = os.path.join(repo_dir, "docs/docs/guides/tracking")
loader = DirectoryLoader(path, glob="**/*.md")
docs = loader.load()
```

```python
# Split the documents into chunks
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
splits = text_splitter.split_documents(docs)

# Wrap the LLM with LangchainLLMWrapper using OpenAI GPT-4 model
evaluator_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o"))

# Generate the test set with the loaded documents (generating 30 examples)
generator = TestsetGenerator(llm=evaluator_llm)

# Assuming the function signature doesn't accept `docs`, pass splits as positional a
# dataset = generator.generate_with_langchain_docs(splits, testset_size=30)
query_distribution = [
    (ComparativeAbstractQuerySynthesizer(llm=evaluator_llm), 0.5),
    (SpecificQuerySynthesizer(llm=evaluator_llm), 0.5),
]

# Call the generate_with_langchain_docs with the custom query_distribution
dataset = generator.generate_with_langchain_docs(
    splits,
    testset_size=30,
    query_distribution=query_distribution
)

# Convert the generated dataset to a Pandas DataFrame
df = dataset.to_pandas()
print(df)

# Optionally, save the generated testset to a CSV file for further inspection
output_csv_path = "generated_testset.csv"
df.to_csv(output_csv_path, index=False)
print(f"Generated testset saved to {output_csv_path}")
```

In RAGAs, the generated test data simulates real-world interactions with a RAG system, providing a structured way to assess both retrieval and generation processes. Each entry in the dataset includes the user input (query), a set of relevant reference contexts from external sources, an ideal reference answer. This data helps evaluate how effectively the system retrieves relevant information and integrates it into accurate, meaningful responses.

A key element of this process is how RAGAs leverages knowledge graphs to enhance

the complexity and realism of the generated queries. Documents are first split into manageable chunks, which are processed using LLM-based extractors to identify key entities, concepts, and relationships. These extracted elements form the nodes of a knowledge graph, while relationships between them—based on shared information or conceptual links—create edges. For example, nodes referencing Einstein and time dilation may be connected by their shared concept. This structured representation allows the system to generate both specific and multi-hop queries that span multiple sources.

By covering a variety of query types—ranging from straightforward fact-based questions to abstract comparisons—the dataset reflects different challenges encountered in real applications.

Step three: Building a RAG pipeline

Now that we have generated a test set, the next step is to build a RAG pipeline using the LangChain framework, which integrates document retrieval and large language models to answer the questions in our test set.

In this setup, LangChain will connect multiple components, including Chroma for vector storage, Anthropic's Claude and OpenAI's GPT-4-mini as LLMs, and OpenAI or Cohere embeddings for document retrieval. The documents will first be embedded into vector representations using either OpenAI or Cohere's models, allowing for efficient similarity searches within the Chroma vector database.

These embeddings convert text into numerical form, making it easier to retrieve relevant documents by matching them to a query. For the two systems, we pair OpenAI's GPT-4o-mini with OpenAI embeddings and Anthropic's Claude 3.5-sonnet with Cohere embeddings.

Here's the code:

```
import os
import nest_asyncio
import pandas as pd
from dotenv import load_dotenv
from langchain_community.document_loaders import DirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
```

```python
from langchain_cohere import CohereEmbeddings
from langchain_anthropic import ChatAnthropic
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnablePassthrough, RunnableLambda
from langchain_core.output_parsers import StrOutputParser
from langchain import hub
import time
# Apply nest_asyncio to avoid event loop issues
nest_asyncio.apply()

# Load API keys
load_dotenv()

anthropic_api_key = os.getenv("ANTHROPIC_API_KEY")
if anthropic_api_key is None:
    raise ValueError("Anthropic API Key not found.")

cohere_api_key = os.getenv("COHERE_API_KEY")
if cohere_api_key is None:
    raise ValueError("Cohere API Key not found.")

# Models to iterate through
gen_models = ["gpt-4o-mini", "claude-3-5-sonnet-20240620"]
embed_models = ["openai", "cohere"]



# Set up the path to your dataset and vector stores
repo_dir = "weave_docs"
if not os.path.exists(repo_dir):
    os.system(f"git init {repo_dir}")
    os.chdir(repo_dir)
    os.system("git remote add origin https://github.com/wandb/weave.git")
    os.system("git sparse-checkout init --cone")
    os.system("git sparse-checkout set docs/docs/guides/tracking")
    os.system("git pull origin master")
    os.chdir("..")

path = os.path.join(repo_dir, "docs/docs/guides/tracking")
loader = DirectoryLoader(path, glob="**/*.md")
docs = loader.load()
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
splits = text_splitter.split_documents(docs)
```

```
# Function to select the appropriate embedding model
def get_embeddings(embed_model):
    if embed_model == "openai":
        return OpenAIEmbeddings()
    elif embed_model == "cohere":
        return CohereEmbeddings(model="embed-english-v3.0")

# Iterate over generation and embedding models
for gen_model, embed_model in zip(gen_models, embed_models):
    print(f"Processing {gen_model} with {embed_model} embeddings...")

    vectorstore_dir = f"vectorstore_{gen_model}_{embed_model}"
    if os.path.exists(vectorstore_dir):
        print(f"Loading vector store from cache: {vectorstore_dir}")
        vectorstore = Chroma(persist_directory=vectorstore_dir, embedding_function=
    else:
        print(f"Creating vector store for {gen_model} and {embed_model}...")
        vectorstore = Chroma.from_documents(documents=splits, embedding=get_embeddi

    retriever = vectorstore.as_retriever()
    prompt = hub.pull("rlm/rag-prompt")

    def format_docs(docs):
        return "\n\n".join(doc.page_content for doc in docs)

    # Capture retrieved contexts
    retrieved_contexts_list = []


    def capture_retrieved_contexts(state):
        """Capture the retrieved contexts and store them for later evaluation."""
        retrieved_docs = state['context']  # Extract the retrieved documents from s

        # If the retrieved_docs are strings, we can directly append them
        if isinstance(retrieved_docs, list):
            retrieved_contexts = [doc if isinstance(doc, str) else getattr(doc, 'pa
        else:
            # If it's a single string or object
            retrieved_contexts = [retrieved_docs if isinstance(retrieved_docs, str)

        retrieved_contexts_list.append(retrieved_contexts)  # Append to the global
        return state  # Pass the state onward
```

```
# Select the appropriate LLM
if "claude" in gen_model:
    model = ChatAnthropic(model=gen_model)
else:
    model = ChatOpenAI(model=gen_model)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | RunnableLambda(capture_retrieved_contexts)
    | prompt
    | model
    | StrOutputParser()
)

# Load and evaluate dataset
file_path = "./generated_testset.csv"
data = pd.read_csv(file_path)
responses = []
retrieved_contexts_list = []


for idx, row in data.iterrows():
    user_input = row['user_input']
    # Add a delay to prevent rate-limiting (e.g., 2 seconds)
    time.sleep(2)  # Adjust the duration based on your API rate limits
    # Generate a response using the RAG chain
    response = rag_chain.invoke(user_input)
    responses.append(response)

# Save results
output_csv_path = f"./results_{gen_model}_{embed_model}.csv"
data['response'] = responses
data['retrieved_contexts'] = retrieved_contexts_list
data.to_csv(output_csv_path, index=False)
print(f"Saved results to {output_csv_path}")
```

The pipeline relies on LangChain to process each query. When a query is submitted, LangChain retrieves the top-ranked documents from the vector store. These documents are passed as context to the LLM via a structured prompt pulled from the

LangChain hub.

To ensure smooth execution, we introduce a short delay between each query to avoid API rate limits. As the pipeline iterates over each query in the test set, it generates a response by integrating the retrieved documents with the LLM's internal knowledge. Both the responses and the retrieved contexts are stored in result files named according to the models used, such as `results_gpt-4o_openai.csv` or `results_claude_cohere.csv`. This organization makes it easy to evaluate the performance of each model combination later on, as well as debug possible issues in the system.

Step four: Evaluating our RAG systems

Now we are ready to evaluate the performance of our RAG pipelines using RAGAs and Weights & Biases.

The evaluation process compares different combinations of generation and embedding models, such as GPT-4o-mini and Claude 3.5, with embeddings from OpenAI and Cohere. Each combination is assessed for its ability to retrieve relevant information, align responses with retrieved data, and produce meaningful answers.

We use RAGAs metrics like factual correctness, faithfulness, answer relevancy, and context recall to analyze how well the models handle retrieval and generation tasks.

Here's the code for evaluation:

```
import pandas as pd
import ast
import os
import numpy as np
import nest_asyncio
from dotenv import load_dotenv
import wandb
from ragas import evaluate
from ragas.llms import LangchainLLMWrapper
from ragas.dataset_schema import EvaluationDataset
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from ragas.metrics import (
    LLMContextRecall, Faithfulness, FactualCorrectness,
    LLMContextPrecisionWithoutReference, NoiseSensitivity,
    ResponseRelevancy, ContextEntityRecall
```

```
)
from ragas.run_config import RunConfig
import plotly.graph_objects as go
import time  # For timestamp-based file naming

# Apply nest_asyncio to avoid event loop issues
nest_asyncio.apply()

# Load environment variables from .env file
load_dotenv()

# Access API keys and validate them
anthropic_api_key = os.getenv("ANTHROPIC_API_KEY")
cohere_api_key = os.getenv("COHERE_API_KEY")
openai_api_key = os.getenv("OPENAI_API_KEY")

if not openai_api_key:
    raise ValueError("OpenAI API Key not found.")
if not anthropic_api_key:
    raise ValueError("Anthropic API Key not found.")
if not cohere_api_key:
    raise ValueError("Cohere API Key not found.")

# Initialize W&B logging
wandb.init(project="RAGAS_Model_Evaluation", name="multi_model_visualization")

# List of generation and embedding models to compare
gen_models = ["gpt-4o-mini", "claude-3-5-sonnet-20240620"]
embed_models = ["openai", "cohere"]

# Helper function to select the appropriate generation model
def get_generation_model(gen_model_name):
    if "claude" in gen_model_name:
        return LangchainLLMWrapper(ChatAnthropic(model=gen_model_name))
    return LangchainLLMWrapper(ChatOpenAI(model=gen_model_name))

# Create a reusable RunConfig
my_run_config = RunConfig(max_workers=1, timeout=180) # preventing rate limiting

# Function to log radar plots
def log_radar_plot(metrics_data, model_pair):
    fig = go.Figure()
```

```
    metrics = list(metrics_data.keys())
    values = list(metrics_data.values())
    metrics += [metrics[0]]  # Close radar loop
    values += [values[0]]

    fig.add_trace(go.Scatterpolar(
        r=values,
        theta=metrics,
        fill='toself',
        name=model_pair
    ))
    fig.update_layout(
        polar=dict(radialaxis=dict(visible=True, range=[0, 1], tickvals=[0, 0.5, 1]
        title=dict(text=f"Radar Plot - {model_pair}", x=0.5, xanchor='center'),
        showlegend=True
    )

    # Save and log radar plot
    timestamp = int(time.time())
    radar_html_path = f"./radar_plot_{model_pair}_{timestamp}.html"
    fig.write_html(radar_html_path, auto_play=False)
    wandb.log({f"Radar Plot {model_pair}": wandb.Html(radar_html_path)})

# Function to log heatmaps
def log_heatmap(heatmap_data, metric_name):
    fig = go.Figure(data=go.Heatmap(
        z=heatmap_data,
        x=["Low", "Medium", "High"],
        y=[f"{gen}_{embed}" for gen, embed in zip(gen_models, embed_models)],
        colorscale='YlGnBu', showscale=True
    ))
    fig.update_layout(
        title=f"{metric_name} Heatmap",
        xaxis_title="Score Bin", yaxis_title="Model Pair"
    )

    heatmap_html_path = f"./heatmap_{metric_name}.html"
    fig.write_html(heatmap_html_path, auto_play=False)
    wandb.log({f"{metric_name} Heatmap": wandb.Html(heatmap_html_path)})

# Helper function to bin metric values
def bin_metric_values(values):
```

```
    bins = [0, 0.3, 0.75, 1.01]
    return np.clip(np.digitize(values, bins) - 1, 0, 2)

# Store heatmap data across models
all_heatmap_data = {metric: [] for metric in ["factual_correctness", "faithfulness"

# Loop through all combinations of generation and embedding models
for gen_model, embed_model in zip(gen_models, embed_models):
    model_pair = f"{gen_model}_{embed_model}"
    output_eval_csv = f"./evaluation_results_{model_pair}.csv"

    # Check if evaluation results exist
    if os.path.exists(output_eval_csv):
        print(f"Loading existing results for {model_pair}.")
        df = pd.read_csv(output_eval_csv)
    else:
        print(f"Running evaluation for {model_pair}...")
        input_csv_path = f"./results_{model_pair}.csv"
        data = pd.read_csv(input_csv_path)

        # Parse retrieved contexts from the CSV
        if 'retrieved_contexts' in data.columns:
            data['retrieved_contexts'] = data['retrieved_contexts'].apply(ast.litera

        # Prepare evaluation dataset
        eval_data = data[['user_input', 'reference', 'response', 'retrieved_contexts
        eval_dataset = EvaluationDataset.from_list(eval_data)

        # Get the appropriate generation model for evaluation
        evaluator_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o-2024-08-06"))

        # Evaluate the dataset
        metrics = [
            LLMContextRecall(), FactualCorrectness(), Faithfulness(),
            LLMContextPrecisionWithoutReference(), NoiseSensitivity(),
            ResponseRelevancy(), ContextEntityRecall()
        ]
        results = evaluate(dataset=eval_dataset, metrics=metrics, llm=evaluator_llm
        df = results.to_pandas()
        df.to_csv(output_eval_csv, index=False)

    # Collect binned values for heatmaps
```

```
    for metric_name in ["factual_correctness", "faithfulness", "answer_relevancy"]:
        binned_values = bin_metric_values(df[metric_name])
        counts = [binned_values.tolist().count(i) for i in range(3)]
        all_heatmap_data[metric_name].append(counts)

    # Log radar plot for the current model pair
    radar_metrics_data = {
        "Context Recall": df["context_recall"].mean(),
        "Factual Correctness": df["factual_correctness"].mean(),
        "Faithfulness": df["faithfulness"].mean(),
        "Context Precision": df["llm_context_precision_without_reference"].mean(),
        "Noise Sensitivity": df["noise_sensitivity_relevant"].mean(),
        "Answer Relevancy": df["answer_relevancy"].mean(),
        "Context Entity Recall": df["context_entity_recall"].mean(),
    }
    log_radar_plot(radar_metrics_data, model_pair)

# Log heatmaps for all metrics across models
for metric_name, heatmap_data in all_heatmap_data.items():
    log_heatmap(heatmap_data, metric_name)

# Finalize W&B logging
wandb.finish()
```

This script first loads the generated responses from the results CSV files and extracts relevant metrics by comparing each generated response against its corresponding reference answer and retrieved contexts. These metrics are then logged into Weights & Biases for easy analysis. This setup allows us to quickly assess how well different RAG pipelines perform and identify areas for improvement.

We specify our desired performance metrics and evaluator model here:

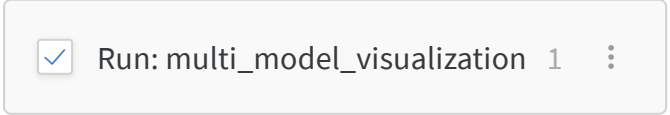
```
evaluator_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o-2024-08-06"))

metrics = [
    LLMContextRecall(), FactualCorrectness(), Faithfulness(),
    LLMContextPrecisionWithoutReference(), NoiseSensitivity(),
    ResponseRelevancy(), ContextEntityRecall()
]
```

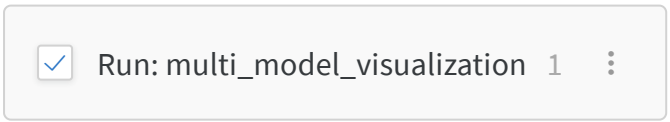
As you can see, RAGAs provides a really nice way to automatically evaluate our

system, without the need to manually implement each metric!

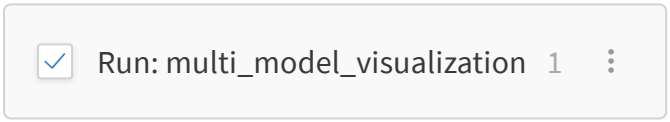
We log the evaluation results through radar plots, which provide a multi-metric comparison for each model combination, and heatmaps to visualize performance trends, which I chose to categorize into 3 categories corresponding to the score given by the evaluator model. This decision was somewhat arbitrary, and you may find different thresholds for performance to be more suitable. To avoid rate limits during API calls, I also made a custom RAGAs RunConfig, which limits parallel workers, which effectively avoids API rate limiting.

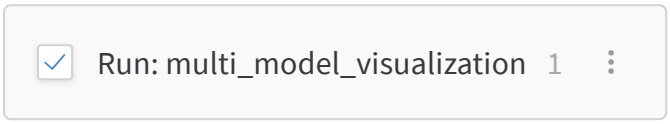
Here are the radar plots for the two systems!

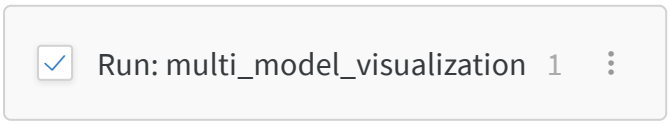




Additionally, here are the heat maps for a few different metrics like factual correctness, answer relevancy, and faithfulness.


Run: multi_model_visualization 1


Run: multi_model_visualization 1


Run: multi_model_visualization 1

Here we can see the GPT-4o mini with OpenAI embeddings outperforms Claude and Cohere on factual correctness and answer relevancy, while Claude with Cohere embeddings wins in terms of faithfulness.

Conclusion

In this tutorial, we explored how RAG systems can enhance large language models by incorporating external data sources to provide more relevant and accurate responses. We introduced RAGAs, a powerful framework that enables the evaluation of RAG pipelines using various metrics, such as Factual Correctness, Faithfulness, Answer Relevancy, and Context Recall.

RAGAs simplifies the evaluation process by automating evaluation data generation, and supporting meaningful metrics. This allows teams to continuously fine-tune their systems, ensuring they remain responsive to user needs and adaptable to new data.

By leveraging Weights & Biases' visualization tools, teams can track performance clearly, identify areas for improvement, and collaborate more effectively. This alignment between evaluation, optimization, and visualization ensures that RAG pipelines not only perform well but also stay reliable across varied scenarios, meeting the demands of real-world use cases.

Recommended Reading

Introduction to RAGAS

Medium

RAGAS Docs

Langchain Docs

RAGAS Paper

Add a comment

Tags: Articles, Intermediate, Tutorial, GenAI, LLM, RAG